Strings Made Up Of Strings

AlgoExpert · Very Hard · Tries

The problem

- Return the strings from `strings` that can be built by concatenating two or more strings from `substrings` (reuse allowed).
- Word-break DP per string.

Intuition

- A trie of the substrings and memoised splitting determine whether a string decomposes into two or more pieces.

Brute force vs optimal

- Try all splits → exponential
- Word-break DP per string → $O(L^2)$ each
- Trie can speed lookups

Approach

- 1. Build a trie of substrings
- 2. From each index, walk the trie for word ends
- 3. Recurse on the rest with memo
- 4. Require at least two pieces

Worked example

```
strings: ['bar', 'are', 'foo', 'ba', 'b', 'barely']

substrings: ['b', 'a', 'r', 'ba', 'ar', 'ely']
```

Which strings are built from ≥ 2 substrings?

Worked example

string	max pieces
bar	3
are	-1
foo	-1
ba	2
b	1
barely	4

Word-break DP per string

Worked example

```
['bar', 'ba', 'barely']
```

Answer

Java solution

```java
public class StringsMadeUpOfStrings {
    public static String[] stringsMadeUpOfStrings(String[] strings, String[] substrings) {
        Set<String> dict = new HashSet<>(Arrays.asList(substrings));
        List<String> out = new ArrayList<>();
        for (String s : strings) {
            int n = s.length();
            int[] pieces = new int[n + 1];
            Arrays.fill(pieces, -1);
            pieces[0] = 0;
            for (int j = 1; j <= n; j++)
                for (int i = 0; i < j; i++)
                    if (pieces[i] >= 0 && dict.contains(s.substring(i, j))) pieces[j] = Math.max(pieces[j], pieces[i] + 1);
            if (pieces[n] >= 2) out.add(s);
        }
        return out.toArray(new String[0]);
    }
}
```

Complexity

- Time: O(n * m^2)
- Space: O(m + s)

Edge cases & pitfalls

- Need ≥ 2 pieces
- Unreachable → −1
- Substring hashing cost

Interview tips & follow-ups

- Same as Concatenated Words

Recap

- A trie of the substrings and memoised splitting determine whether a string decomposes into two or more pieces.
- Complexity: O(n * m^2)